Uploads, WordPress Sizes and Lossless-Grade Compression

a cookbook for hkm-plugin-image

File upload • Base64 • WordPress sizes • Compression

AlfacodeTeam PhpServicePlatform — `image.processing`

Every number recorded from real code via `docs/generators/probe_usage.php`

Engines on this run: GD, Imagick, Vips

Contents

1 What this guide covers **2**

2 Uploading a normal file **2**

3 Uploading base64 / a data URI **3**

4 Resizing like WordPress **3**

5 Compressing without losing quality **4**

5.1 Measured quality vs. size vs. fidelity (PSNR) 5

5.2 `jpeg` 5

5.3 `webp` 5

5.4 `avif` 5

5.5 `stripMetadata()` and file size 6

6 Putting it together **6**

1 What this guide covers

Four things people building an upload endpoint on `hkm-plugin-image` ask for first: taking a normal multipart file upload, taking a base64 / data-URI upload (the shape a `<canvas>` or a JS cropper library sends), generating a WordPress-style set of named sizes, and compressing without a visible quality loss. This guide is a cookbook for exactly those four, not a repeat of the full method reference.

Every number here was recorded, not written

`docs/generators/probe_usage.php` runs the REAL kernel `UploadedFile` class end to end, a real base64/data-URI decode, real `resize()`/`variants()` calls, and a real measured PSNR sweep across GD, Imagick, Vips on a generated 900x600 test image. Regenerate with `php probe_usage.php && python3 build_usage.py`.

2 Uploading a normal file

This plugin has no HTTP layer of its own — it is a pure byte-string transform (`API/Contracts/ImageProcessorContract`). Reading the upload is the kernel's job: `Request::file()` returns a `?UploadedFile`, and `UploadedFile::contents()` reads the temp file into a string, which is exactly what every `ImageProcessorContract` method takes.

```
final class AvatarController extends ApiController        // RequestAware
{
    public function __construct(
        private readonly ImageProcessorContract $image,
        private readonly StoragePort $storage,
    ) {}

    public function upload(): Response
    {
        $file = $this->request->file('avatar');
        if ($file === null || !$file->isValid()) {
            return Response::unprocessable(['avatar' => 'A valid image file is required.']);
        }

        $bytes = $file->contents();   // reads the temp upload into a string

        $this->image->validate(
            $bytes,
            maxBytes: (int) image_config('max_bytes'),
            allowedMimeTypes: image_allowed_mime_types(),
        );

        $stripped = $this->image->stripMetadata($bytes);   // drop GPS/camera EXIF before storing
        $path     = $this->storage->store($stripped, $file->clientName(), 'uploads/avatars');

        return Response::json(['path' => $path], 201);
    }
}
```

Recorded — the real UploadedFile round-trip

A real `Symfony\UploadedFile` was written to a real temp file, promoted via `Kernel\Http\UploadedFile::createFromBase()` (the same call `DomainResolver`'s request pipeline makes), then read with `contents()`. Client name recorded: `holiday.jpg`, reported size `276341` bytes, `contents()` returned 276341 bytes — an exact byte-for-byte match to the source (yes) — and `ImageProcessorContract::dimensions()` decoded it successfully (yes).

`$file→isValid()` catches a failed/partial upload before you touch the bytes at all. `validate()` still needs to run explicitly for the per-route MIME allowlist — but even if you forget it, every engine's `decode()` now enforces `IMAGE_MAX_BYTES`/`IMAGE_MAX_WIDTH`/`IMAGE_MAX_HEIGHT` structurally, so a skipped `validate()` no longer opens an unbounded decompression-bomb path.

3 Uploading base64 / a data URI

A JS cropper (`react-easy-crop`, `cropperjs`, a `<canvas>.toDataURL()` call) sends a JSON body with a `....` string, not a multipart file. Stripping the `data:...;base64,` prefix (if present) and decoding is the ONLY extra step — `ImageProcessorContract` doesn't care where the bytes came from.

```
final class DecodeBase64Image
{
    public static function fromRequest(Request $request, string $field = 'image'): string
    {
        $input = (string) $request->input($field, '');
        if (str_starts_with($input, 'data:')) {
            $comma = strpos($input, ',');
            $input = $comma === false ? '' : substr($input, $comma + 1);
        }

        $decoded = base64_decode($input, strict: true);
        if ($decoded === false) {
            throw new \DomainException('Image: not valid base64.');
        }
        return $decoded;
    }
}

// In the controller:
$bytes = DecodeBase64Image::fromRequest($request, 'avatar_base64');
$this->image->validate($bytes, maxBytes: (int) image_config('max_bytes'), allowedMimeTypes: image_allowed_mime_types());
```

Recorded

A real `...` string (368479 chars) was built from the same 900x600 test image, decoded with the function above, and matched the original bytes exactly (yes) with correct dimensions (yes). A raw base64 body with no `data:` prefix at all decoded identically (yes). Garbage input (`%%%not-base64%%%`) was rejected with a `\DomainException` — base64_decode(strict) returned false before any image call ran — before any image engine ever saw a byte.

`base64_decode($input, strict: true)` is the important part: WITHOUT strict mode, PHP silently skips invalid characters instead of failing, which can turn a corrupted upload into a corrupted-but-"successfully decoded" byte string. `strict: true` returns `false` on the first invalid character, so the `\DomainException` above fires before that garbage ever reaches an image engine.

4 Resizing like WordPress

WordPress's default sizes are `thumbnail` (150x150, HARD-cropped to an exact square), `medium` (300px box, soft-fit, no crop), `medium_large` (768px wide, proportional), `large` (1024px box, soft-fit, no crop) and the original (`full`). Mapped onto this contract's `ResizeMode`: `thumbnail`'s hard crop is `Cover`; `medium`/`medium_large`/`large`'s soft, never-upscaling fit is `Contain`.

The one thing that trips people up: variants() presets are WIDTH ONLY

`variants($contents, array $presets, ...)` takes `array<string, int>` — a NAME to WIDTH map, with no per-preset height. Even in `Cover` mode, the height is computed PROPORTIONALLY from the source's own aspect ratio, not forced to a square. Recorded proof: `variants($photo, ['thumbnail' => 150, ...], mode: Cover)` on this 900x600 (3:2) source produced a `150×100` thumbnail — NOT `150×150`. For a true WordPress-style square thumbnail, call `resize()` directly with BOTH dimensions: `resize($contents, 150, 150, ResizeMode::Cover)` — recorded output: `150×150`, confirmed square.

```
// The hard-cropped, exact-square thumbnail — a SEPARATE call, explicit height.
```

```php
$thumbnail = $image->resize($bytes, 150, 150, ResizeMode::Cover);

// The soft-fit, proportional sizes — ONE variants() call, WIDTH only, never upscales.
$sizes = $image->variants(
    $bytes,
    presets: ['medium' => 300, 'medium_large' => 768, 'large' => 1024],
    format: ImageFormat::Webp,
    mode: ResizeMode::Contain,
);

// "full" — keep the original, just strip metadata and re-orient it.
$full = $image->orient($image->stripMetadata($bytes));

foreach (['thumbnail' => $thumbnail, ...$sizes] as $name => $result) {
    $contents = $result instanceof ImageVariant ? $result->contents : $result;
    $storage->store($contents, "{$name}.webp", "uploads/{$id}");
}
$storage->store($full, "full.jpg", "uploads/{$id}");
```

Size	Call	Recorded output
thumbnail	`resize($, 150, 150, Cover)`	150x150 (7.4 KB)
medium	`variants()` preset 300, `Contain`	300x200 (16.1 KB)
medium_large	`variants()` preset 768	768x512 (36.4 KB)
large	`variants()` preset 1024	900x600 (51.7 KB)

Recorded — never upscales, same as WordPress

Requesting `resize($bytes, 1024, 1024, ResizeMode::Contain)` on a 200x120 source (smaller than the 1024 target) came back `200×120` — the ORIGINAL size, not stretched up. Only `ResizeMode::Stretch` upscales, and only because an exact size was asked for explicitly.

5 Compressing without losing quality

”Lossless” means two different things depending on the format, and it matters which one you’re asking for:

Format	Through this plugin
PNG	ALWAYS pixel-lossless, at every quality setting — `quality` maps to PNG’s compression LEVEL (speed/size trade-off), never to visual fidelity
JPEG / WebP / AVIF	always LOSSY through `convert()`/`resize()`/`variants()` — even at `quality: 100` — no true bit-exact lossless mode is exposed by this contract for these formats

Recorded — PNG really is pixel-identical at every quality setting

Encoded the same source as PNG at quality 10, 50, 82, 100 — every single one came back pixel-identical to the source over the sampled grid (`psnr_db: null` = infinite/lossless). Only the byte size changes: 10 -> 332.4 KB, 50 -> 335.8 KB, 82 -> 393.8 KB, 100 -> 1585.2 KB.

5.1 Measured quality vs. size vs. fidelity (PSNR)

PSNR (peak signal-to-noise ratio) is an objective measure of how close the re-encoded pixels are to the original — higher is more faithful; above roughly 40 dB is usually indistinguishable to the eye, 30-40 dB is generally acceptable, below 25 dB is often visibly soft or blocky. Measured here on a deliberately CHALLENGING test image (fine noise texture plus hard-edged shapes, not a smooth photo) — treat these as a conservative floor: an ordinary photograph without literal per-pixel noise will typically reach the same or higher PSNR at the SAME or a LOWER quality setting.

5.2 `jpeg`

Quality	Size	vs. source	Measured PSNR
60	49.9 KB	18.5% of source	23.42 dB
75	64.8 KB	24% of source	24.03 dB
85	88.2 KB	32.7% of source	24.42 dB
90	169.6 KB	62.8% of source	36.99 dB
95	253.5 KB	93.9% of source	44.12 dB
100	462.5 KB	171.4% of source	60.73 dB

5.3 `webp`

Quality	Size	vs. source	Measured PSNR
60	38.0 KB	14.1% of source	24.36 dB
75	43.1 KB	16% of source	24.43 dB
85	56.5 KB	20.9% of source	24.65 dB
90	67.4 KB	25% of source	24.74 dB
95	86.0 KB	31.9% of source	24.76 dB
100	112.2 KB	41.6% of source	24.8 dB

5.4 `avif`

Quality	Size	vs. source	Measured PSNR
60	37.7 KB	14% of source	24.2 dB
75	47.3 KB	17.5% of source	24.27 dB
85	61.2 KB	22.7% of source	24.3 dB
90	153.4 KB	56.9% of source	46.39 dB
95	195.4 KB	72.4% of source	48.73 dB
100	345.9 KB	128.2% of source	lossless (pixel-identical)

What the measured curve says for THIS content

Quality 60-85 sat in a comparatively low, fairly flat fidelity band across all three lossy formats here; fidelity jumped sharply at quality 90 and kept climbing to 95/100. For content with fine texture or sharp edges, don't assume "quality 85" is close to lossless by default — measure it on your own real images. `webp` and `avif` reached a SMALLER file size than `jpeg` at a comparable measured PSNR in this run — but that comparison is specific to this test image and this build's encoders, not a universal claim.

Practical recommendation from this data: for "no visible quality loss," use `IMAGE_QUALITY=90` or

higher with `webp` or `avif` — both landed meaningfully smaller than `jpeg` at a similar or better measured PSNR in this run. If you need a GUARANTEE of zero pixel loss rather than "very high fidelity," the only option through this contract is PNG (or storing the untouched original bytes) — not a high JPEG/WebP/AVIF quality number, however close to 100.

5.5 `stripMetadata()` and file size

A fake but realistically-sized EXIF block (~2677 bytes — a `UserComment` tag padded to look like typical camera/GPS metadata) was spliced into a real JPEG with no other change. That is the metadata's TRUE byte contribution to the file: `276341` bytes plain vs. `279018` bytes with the block attached.

Recorded — every engine already strips metadata on every operation

`stripMetadata()`'s own output (204.1 KB) was BYTE-IDENTICAL to a plain `convert()` at the same fixed quality with the EXIF block still attached (204.1 KB) — measured savings from calling `stripMetadata()` specifically, beyond what any other transform already does: 0 bytes. That matches the documented behaviour: every engine strips EXIF/IPTC/XMP on EVERY encode, not only when you call `stripMetadata()`. So the metadata overhead disappears the moment you call ANY method here — `stripMetadata()` exists specifically for when you want it gone WITHOUT also resizing.

6 Putting it together

```
$file  = $request->file('photo');
$bytes = $file->contents();

$image->validate($bytes, maxBytes: (int) image_config('max_bytes'), allowedMimeTypes: image_allowed_mime_types());

$upright = $image->orient($bytes);                                  // fix sideways phone photos first
$thumb   = $image->resize($upright, 150, 150, ResizeMode::Cover);   // WP-style hard-cropped thumbnail
$sizes   = $image->variants(                                        // WP-style soft-fit sizes, high quality, small format
    $upright,
    presets: ['medium' => 300, 'medium_large' => 768, 'large' => 1024],
    format: ImageFormat::Webp,
    quality: 92,                                                    // see "Compressing without losing quality" — 90+ is where
     fidelity gets high
    mode: ResizeMode::Contain,
);
$full = $image->stripMetadata($upright);                            // original size, GPS/camera data gone

$storage->store($thumb, 'thumbnail.jpg', "uploads/{$id}");
foreach ($sizes as $name => $variant) {
    $storage->store($variant->contents, "{$name}.webp", "uploads/{$id}");
}
$storage->store($full, 'full.jpg', "uploads/{$id}");
```

A route wiring this up must still declare `"requires": ["image.processing"]` in its `module.json` — the contract is not bound otherwise.